Enterprise Role-Based Access Control and Identity Auditor: An Explainable Risk-Based Framework for Privilege Auditing

P. Varun Reddy · K. Chandana · Pardhava Nandan

Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Hyderabad, India

Under the guidance of Dr. K. Swanthana

Abstract—Role-based access control (RBAC) simplifies enterprise authorization by associating users with roles and roles with permissions. However, static role assignments can outlive business needs, resulting in unused privileges, excessive access, and privilege creep. This paper presents the design and reported prototype functionality of an Enterprise RBAC and Identity Auditor that integrates identity administration, access enforcement, security-event logging, privilege auditing, and explainable risk scoring. The proposed approach examines privilege level, excess and unused permissions, device and network context, login time, failed authentication, suspicious activity, repeated unauthorized access, and high-risk role assignments. A rule-based score supports four audit-priority levels and actionable least-privilege reviews. The project reviews describe a Java/Spring Boot, Spring Security, JWT, React.js, and MySQL implementation with user registration, RBAC mappings, logging, and access decisions. This paper specifies reproducible evaluation scenarios but does not claim measured accuracy or runtime improvements because supporting experiment logs and numeric results were not supplied. The contribution is an auditable framework for linking authorization decisions with continuous identity-governance review.

Keywords—Role-Based Access Control, Identity Auditing, Privilege Creep, Least Privilege, Risk Scoring, Spring Security, Enterprise Security.

I. INTRODUCTION

Enterprise information systems must protect applications and records used by employees with different responsibilities. RBAC models authorization through user–role and role–permission assignments, reducing the complexity of assigning permissions individually [1,2]. Its effectiveness depends on maintaining assignments as job duties change. When old permissions remain active, privilege creep creates unnecessary exposure. An identity auditor supplements access enforcement by examining permissions, account activity, and deviations from access policy. The present project integrates administration, auditing, explainable scoring, and review recommendations in one workflow.

II. PROBLEM STATEMENT AND RESEARCH OBJECTIVES

Conventional RBAC can correctly enforce an assigned permission while failing to determine whether the assignment is still appropriate. The resulting governance problem includes excessive privileges, dormant privileges, conflicting role assignments, and repeated access-policy violations. The proposed system aims to manage identities, roles and permissions; log authentication and authorization events; detect suspicious or unnecessary access; prioritize reviews using interpretable risk rules; and produce actionable reports.

Research questions: (RQ1) How can privilege-audit signals be combined with RBAC enforcement? (RQ2) Can a deterministic risk score prioritize identity reviews transparently? (RQ3) How can test scenarios verify access enforcement and privilege-creep alerts without conflating risk scores with predictive accuracy?

III. RELATED WORK AND RESEARCH GAP

The foundational RBAC model formalizes the relationship among users, roles, and permissions [1]. The NIST RBAC model extends this with role hierarchies and constraints [2]. The principle of least privilege encourages granting only access needed for assigned duties [3]. NIST guidance on access control and audit accountability motivates periodic review and logging [4]. The project's Review-1 literature survey additionally identifies role-mining research on constraints, incremental updates, graph representations, and scalable role generation; the slide bibliography should be independently verified before those individual papers are cited in a publication.

Role mining primarily helps derive or refine role structures; access enforcement determines whether a request matches a configured policy. Neither alone guarantees ongoing examination of stale or excessive assignments. The project therefore targets a practical integration gap: combining RBAC management, privilege auditing, event monitoring, explainable risk categorization, and least-privilege recommendations in one prototype. This is a proposed integration, not a claim that no previous systems have combined these features.

IV. PROPOSED ARCHITECTURE AND METHODOLOGY

The workflow described in Project Review-2 is: User → Authentication → RBAC → Audit → Risk Analysis → Report. The authentication layer validates identities; the RBAC layer checks the effective permissions derived from user–role and role–permission mappings; audit services record relevant events; a privilege-audit service identifies excess or unused assignments; the risk service scores detected conditions; and a reporting service presents violations and suggested review actions.

4.1 Core services

4.2 Identity and privilege audit

A user's effective permission set can be defined as $P(u) = \cup\{P(r): r \in R(u)\}$, where $R(u)$ denotes assigned roles and $P(r)$ the permissions of a role. An excessive privilege is a granted permission that is not justified by an approved job requirement; an unused privilege is a permission with no recorded use over the defined observation window (30 days in the project review). Unused access should trigger review rather than automatic removal, since lack of use alone does not prove that access is unnecessary. Potential conflicts should be checked against organization-defined separation-of-duties constraints.

4.3 Explainable risk assessment

For each evaluated identity or access context, let x_i be 1 if risk condition i applies and 0 otherwise, with the weights w_i listed below. Define $S = \min(100, \Sigma\, w_i x_i)$. The cap at 100 is a proposed

normalization choice for the paper; the slides specify the factor weights and score bands but do not explicitly state a maximum. The score is a policy-priority indicator, not a calibrated probability of compromise.

The review slide specifies +5 for one failed attempt but does not define the escalation function for two or more attempts; therefore, no additional failed-attempt weights are assumed here. Conditions such as “suspicious session” and “excessive permissions” require organization-specific detection rules before experimental replication.

Authorization and audit priority are separate outputs. A request without an assigned permission must be DENIED regardless of a low risk score. Conversely, a high risk score does not prove malicious behavior: it should trigger the review or restriction prescribed by policy. Project Review-2 reports ALLOW/DENY decisions, not a fully specified conditional-access enforcement protocol.

The following figures summarize the project design and the rule weights presented in the review slides. Conceptual diagrams are architectural illustrations, while the risk scenario chart contains worked examples rather than experimentally observed outcomes.

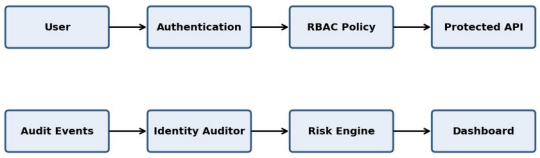


Fig. 1. Proposed enterprise RBAC and identity-auditing architecture.



Fig. 2. User–role–permission relationships in the RBAC model.



Fig. 3. Identity auditing and privilege-review workflow.

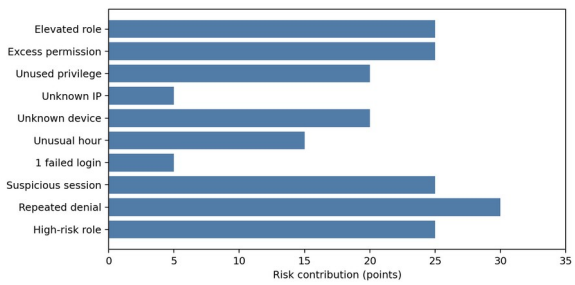


Fig. 4. Risk contributions specified in Project Review-2.

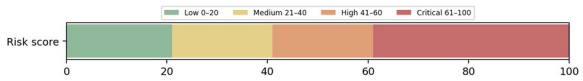


Fig. 5. Risk classification thresholds from Project Review-2.

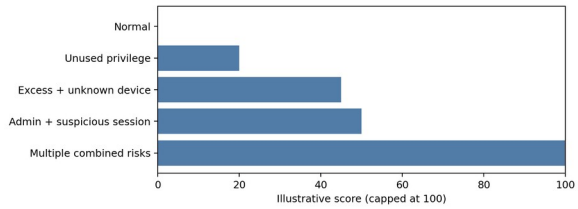


Fig. 6. Illustrative risk scores calculated from the proposed rules (not measured results).

V. IMPLEMENTATION ENVIRONMENT

Project Review-2 identifies Java, Spring Boot, Spring Security, JWT, BCrypt, Spring Data JPA/Hibernate, REST APIs, React.js, MySQL, Maven, and Postman. The slide describes Windows 10/11 as the development environment. Review-1 contains an earlier alternative stack (Python/FastAPI and PostgreSQL/MongoDB); this paper follows the more recent Review-2 stack, which should be confirmed against the final source code before submission.

The Review-2 presentation lists user registration, secure login, password hashing, role and permission assignment, user–role–permission mapping, privileged-account monitoring, privilege-creep detection, least-privilege monitoring, identity auditing, event logging, risk classification, and ALLOW/DENY access decisions as current features. These are project-team claims from the slides, not independently verified code-execution results.

VI. EXPERIMENTAL DESIGN

Evaluation should use a controlled database of test users, roles, permissions, access events, and documented expected outcomes. Test cases should include valid and invalid credentials; authorized and unauthorized resource access; changed role assignments; stale permissions; a privileged account with excessive rights; unusual device or login-time context; and repeated access denial. For each case, record inputs, expected decision, observed decision, score components, audit-log entry, and execution time. Tests must be repeated after restarts to establish whether event history persists.

The combined score of 35 in the example is calculated from the published Review-2 weights, not an observed application result. The unauthorized-access trigger requires a defined repetition threshold, which the slides do not provide.

VII. RESULTS AND EVALUATION STATUS

The project presentation identifies functional modules and describes testing categories, including registration/login, role assignment, RBAC authorization, unauthorized access, privilege-creep detection, and security logs. However, the Results slides do not provide extractable numeric performance tables, test counts, measured latency, precision/recall, or verified experimental outcomes. Accordingly, this draft does not report fabricated empirical measurements.

The rule table supports deterministic illustrative calculations: an unknown device contributes 20 points (Low); a privileged user with excessive permissions contributes 50 points (High); an unknown device plus an unusual login time contributes 35 points (Medium); and an elevated account with excess permissions and

an unused privilege contributes 70 points (Critical). These examples verify arithmetic against the stated policy only. They do not establish that the implemented software produced these scores.

VIII. DISCUSSION

Combining RBAC with identity auditing addresses a different question from ordinary authorization: not only “may this user access the resource?” but also “should this user continue to hold this privilege?” Transparent factor-level scoring makes prioritization easy to explain to administrators. The selected weights are heuristic and may over-prioritize legitimate administrators or after-hours workers. Organizations should calibrate weights, evaluate false positives, define permission-usage evidence, and require human review for high-impact revocations.

IX. LIMITATIONS AND THREATS TO VALIDITY

The supplied materials do not include application source code, raw test logs, dataset size, hardware specifications, or comparative benchmarks. Thus implementation completeness and empirical effectiveness cannot be independently established from these documents. Rule weights and cutoffs have not been statistically calibrated. “Excessive” permissions require approved job-role baselines, and “unused” permissions require reliable activity telemetry. Device/IP signals can change legitimately. A single scoring model may not generalize across organizations, and audit records require retention, access protection, and tamper resistance.

X. CONCLUSION AND FUTURE WORK

This paper presents an integrated enterprise RBAC and identity-auditing design based on the two project reviews. It connects role/permission administration with event logging, privilege-creep review, explainable risk assessment, and least-privilege recommendations. The four-level score supports understandable audit prioritization while RBAC remains the primary permission-enforcement mechanism. Future work should validate the running prototype with reproducible datasets, publish measured test results, persist and protect audit evidence, formalize conflict and repeated-failure rules, compare against a static RBAC baseline, and evaluate usability and false-positive rates.

REFERENCES

[1] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[2] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” ACM Transactions on Information and System Security, vol. 4, no. 3, pp. 224–274, 2001.

[3] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975.

[4] NIST, Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5, 2020.

[5] Team 16, “Enterprise Role-Based Access Control & Identity Auditor,” Project Review-1 presentation, Koneru Lakshmaiah Education Foundation, 2026 (project source).

[6] Team 16, “Enterprise Role-Based Access Control (RBAC) & Identity Auditor,” Project Review-2 presentation, Koneru Lakshmaiah Education Foundation, 2026 (project source).